

Лабораторная работа №13

XML DTD

Цель работы: познакомиться DTD определениями типа XML документов, научиться описывать структуру XML документов.

Теоретические сведения

Можно сказать, что XML документы состоят из элементов и атрибутов. В DTD описываются и некоторые другие объекты, определения которых смотрите ниже, но документы всегда поддерживают эти две основные концепции.

По отношению к документу DTD могут быть *внутренние* и *внешние*. Внутренние DTD описываются непосредственно в файле XML документа вот так:

```
<?xml version="1.0" ?>
<!DOCTYPE queue [
    ...
]>
<queue>
<message to="aaa@bbb.cc" from="ccc@bbb.aa" subj="subject of ...">
    Text of mail message ...
    ...
</message>
```

Строка **<!DOCTYPE** queue [называется *определением типа документа*. За ключевым словом **DOCTYPE** должно следовать имя корневого элемента XML документа (в примере - **queue**). Далее, в квадратных скобках следует список *элементов, атрибутов и сущностей* - именно из этих компонентов состоит XML документ. Заканчивает DTD последовательность **]>**.

DTD может также располагаться и во внешнем файле. Внешние DTD делятся на системные (SYSTEM) и общедоступные (PUBLIC). В любом случае файл DTD должен иметь расширение **.dtd**. Связывание системного DTD и XML документа выглядит следующим образом:

```
<?xml version="1.0" ?>
<!DOCTYPE queue SYSTEM "./dtd/msg_queue.dtd">
<queue>
    ...
</queue>
```

В данном случае анализатору указывается, что DTD находится во внешнем файле, по адресу **./dtd/msg_queue.dtd**. Существует ряд общеизвестных DTD, разработанных для определенных целей. В случае использования такого DTD его объявление будет иметь несколько другой вид:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd" >
```

На то, что DTD является общеизвестным, указывает ключевое слово *PUBLIC*. Рассмотрим подробнее строку `"-//W3C//DTD XHTML 1.1//EN"`. Если DTD является стандартом ISO, то данная строка начинается словом ISO. Если DTD не является стандартом ISO, но используемый стандарт был принят группой стандартизации, то DTD начинается со знака (+). Если же он не был принят официально группой стандартизации, то объявление следует начинать со знака (-). Далее, с разделителем (//) следуют владелец данного DTD, имя DTD и язык. Из приведенного примера видно, что используется DTD, не стандартизованное официально, принадлежащее компании W3C, с именем *XHTML 1.1*.

Приведем несколько доводов в пользу внешних DTD:

- Внешние DTD могут быть общеизвестными (public).
- Внешние DTD позволяют отделить структуру от содержания. Вы можете изменить и расширить правила словаря, не открывая содержания XML документов.
- Можно однажды написанное DTD использовать во множестве документов.

Далее подробно рассматриваются элементы DTD.

Сущности

В языке XML есть возможность продекларировать фрагменты содержания, а затем ссылаться на них при необходимости, что позволяет сэкономить время и силы разработчикам. Объявляя сущность в DTD мы определяем ее имя и содержание, на которое она ссылается. Ссылаясь на сущность, мы заставляем анализатор заменить ссылку на содержимое сущности. Сущности бывают

- *анализируемыми* (parsed entity) - содержимое сущности анализируется, т.е, например, при встрече в нем ссылок на другие сущности, они так же замещаются их содержимым; разметка внутри таких сущностей так же интерпретируется.
- *неанализируемыми* (unparsed entity) - это не обязательно текст в XML формате, или совсем не текст (графика или другое мультимедиа содержание).

Ниже приведен пример объявления и использования анализируемых сущностей **author** и **copyright** (обратите внимание: сущность **copyright** ссылается на **author**):

```
<?xml version="1.0" ?>
<!DOCTYPE lesson [
  <!ENTITY author "Иванов Иван Иванович">
  <!ENTITY copyright "2003, Odessa, &author;">
]>
<lesson>
  <theme>Document Type Definition - DTD</theme>

  <author>&author;</author>
  В данном примере мы с вами рассмотрим ...
  ...
  <copyright>&copyright;</copyright>
</lesson>
```

Строка `<!ENTITY author "Иванов Иван Иванович">` объявляет сущность **author** со значением "Иванов Иван Иванович". Ссылка на сущность выглядит как символьная подстановка в HTML: `&author;`. При разборе документа анализатор встретит ссылку на сущность и заменит ее на значение сущности. В итоге документ примет следующий вид:

```

<?xml version="1.0" ?>
<!DOCTYPE lesson [
  <!ENTITY author "Иванов Иван Иванович">
  <!ENTITY copyright "2003, Odessa, &author;">
]>
<lesson>
  <theme>Document Type Definition - DTD</theme>

  <author>Иванов Иван Иванович</author>
  В данном примере мы с вами рассмотрим ...
  ...
  <copyright>2003, Odessa, Иванов Иван Иванович</copyright>
</lesson>

```

Из приведенного рисунка видно, что ссылка на сущность *&author;* была заменена ее содержимым "Иванов Иван Иванович", причем и в "теле" другой сущности - *©right;*. Анализируемые сущности могут также содержать и разметку.

Внешние сущности

Текст замещения может содержаться во внешнем файле, тогда объявление сущности будет выглядеть так:

```

<!DOCTYPE doc [
  <!ENTITY license SYSTEM "./files/license.txt"
...
]>
<license>
  &license;
</license>

```

Вместо ключевого слова *SYSTEM* может быть использовано слово *PUBLIC*, в случае, если речь идет об общедоступной сущности (см. системные и общедоступные DTD).

В XML существует ряд предопределенных сущностей, служащих для преодоления ограничений, связанных с синтаксисом языка. Перечень их приведен в таблице ниже:

Символ	Ссылка на сущность
>	<
<	>
&	&
"	"
' (апостроф)	'

Таким образом, для вывода текста

```
if ((a < b)&&(c > 150)) { ... }
```

нужно использовать следующий код:

```
if ((a &lt; b)&amp;&amp;(c &gt; 150)) { ... }
```

Кроме того, на любой символ можно сослаться по его коду в таблице символов. Например, символу '<' соответствует ссылка '<' (десятичный код) или '<' (16ричный код);

Параметрические сущности

Данный подвид сущностей относится к анализируемым и предназначен для использования строго внутри DTD. Они дают возможность сослаться на часто встречающиеся в DTD конструкции, что впоследствии позволяет их быстро редактировать. Для объявления таких сущностей используется ключевое слово **ENTITY**, знак %, имя и значение сущности.

```
<!ENTITY % peopleParams "name CDATA #REQUIRED age CDATA #IMPLIED  
country CDATA #REQUIRED" >
```

Демонстрация ссылки на такого рода сущность потребует знаний других элементов DTD, поэтому их описание приведено ниже.

Неанализируемые сущности и нотации

Нотации (**NOTATION**) описывают содержание, разработанное не на языке XML. Используются они для того, чтобы объявить конкретный класс данных и связать его с внешней программой. Эта внешняя программа становится обработчиком объявленного класса данных. Например, связав с документом изображение в формате **JPEG**, разработчик желает, чтобы программа приняла и визуализировала двоичные данные в этом формате. Ниже представлены примеры объявления нотаций:

```
<!NOTATION doc SYSTEM "winword.exe">  
<!NOTATION jpeg SYSTEM "c:\Progrm Files\ACDSee\acdsee.exe">
```

Где же используются нотации? Например, для объявления двоичных сущностей:

```
<!ENTITY photo SYSTEM "images/myphoto.jpg" NDATA jpeg>
```

На то, что сущность является неанализируемой, двоичной указывает ключевое слово **NDATA** - *notation data, данные нотации*, за которым следует имя объявленной ранее нотации.

Элементы

"Сердцем и душой" документа XML является элемент. В DTD типы элементы объявляются при помощи ключевого слова **ELEMENT**:

```
<!ELEMENT message (#PCDATA)>  
...
```

За ключевым словом **ELEMENT** должно следовать имя элемента (message), затем, спецификация содержания (**#PCDATA** - анализируемые символьные данные). Существуют следующие варианты описания содержания элементов:

- **EMPTY**
- **ANY**
- (**#PCDATA**)

- (модель содержания)
EMPTY

При помощи ключевого слова **EMPTY** описываются элементы с пустым содержанием. Такие элементы не могут содержать текст или порожденные (дочерние) элементы, но могут иметь атрибуты. Например:

```
<!ELEMENT logo EMPTY>
...
<logo src="/images/logo.gif" address="http://company.com"/>
```

Подобные элементы наверняка встречались вам и ранее в курсе HTML. Например:

```
<br />

```

#PCDATA - анализируемые символьные данные

Такая спецификация содержания элемента указывает на то, что элемент может содержать только текст (возможно, со ссылками на сущности - это и делает его анализируемым...). В содержимом таких элементов недопустимо использование других элементов:

```
<!ELEMENT author (#PCDATA)>
...
<!-- корректная конструкция -->
  <author>Автор: &author;</author>
<!-- некорректная конструкция (содержит элемент <b>) -->
  <author><b>Автор:</b> &author;</author>
```

ANY

Данный тип содержания полностью соответствует значению слова: **ANY** - любой. Элемент может содержать анализируемые текстовые данные (**#PCDATA**) вперемешку с дочерними элементами. Например:

```
<!ELEMENT lesson ANY>
...
<lesson>
  Название: <title>Использование DTD</title>
  Автор: <author>&author;</author>
  ...
</lesson>
```

Чисто элементное содержание

Иногда возникает необходимость задать структуру элемента, содержащего некоторые дочерние элементы, более жестко, чем это позволяет тип **ANY**. В таком случае нам придется воспользоваться некоторыми спецсимволами для описания модели содержания элементов:

Символ	Значение
	Задаёт выбор одного варианта из нескольких возможных: <pre><!ELEMENT elem (a b c)></pre>

	означает, что описываемый элемент (elem) может содержать или элемент a , или элемент b , или элемент c .
,	Указывает, что перечисленные элементы должны следовать в заданном порядке: <pre><!ELEMENT elem (a,b,c)></pre> означает, что описываемый элемент содержит элемент a, затем элемент b и, наконец, элемент c.
?	Указывает, что перечисленные элементы необязательны, но, если присутствуют, должны использоваться только 1 раз: <pre><!ELEMENT elem (a,(b c)?)></pre> означает, что описываемый элемент содержит элемент a, затем необязательный элемент b или необязательный элемент c.
*	Указывает, что перечисленные элементы могут повторяться неограниченное количество раз, или ни разу: <pre><!ELEMENT elem (a*)></pre> означает, что описываемый элемент может быть пустым, или содержать несколько элементов a.
+	Указывает, что перечисленные элементы могут повторяться неограниченное количество раз (минимум 1 раз): <pre><!ELEMENT elem (a*)></pre> означает, что описываемый элемент может содержать один или более элементов a.
()	Скобки служат, как вы уже наверное догадались, для группировки элементов.

Смешанное содержание

Когда необходимо указать в качестве содержания элемента набор определенных дочерних элементов и символьных данных используют следующий подход:

```
<!DOCTYPE formatted-text [
  <!ELEMENT strong (#PCDATA)>
  <!ELEMENT emphasis (#PCDATA)>
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT paragraph (#PCDATA|strong|emphasis)*>
  <!ELEMENT section (title,paragraph+)>
]>
```

Обратите внимание на описание элемента **paragrapg**: он может содержать неограниченную (возможно, пустую) последовательность элементов **strong** или **emphasis** или символьные данные. Ниже приводится пример применения этих элементов:

```

<section>
  <title>Смешанное содержание</title>
  <paragraph>
    Как следует из самого названия, смешанное содержание
    (<emphasis>mixed content</emphasis>) является сочетанием
    элементного содержания и символьных данных.
    ...
  </paragraph>
  ...
</section>

```

Атрибуты

По существу, атрибуты обеспечивают дополнительную информацию об элементах XML и их содержании. Если элемент является пустым, то атрибуты несут дополнительное содержание. Если в элементе есть содержание, атрибуты, как правило, описывают его.

В DTD атрибуты описываются при помощи декларации "<!ATTLIST " (список атрибутов). Следом за ключевым словом **ATTLIST** следует имя элемента, к которому применяется список атрибутов, а затем 0 или более определений атрибутов. Каждое определение атрибута состоит из имени атрибута, его типа и характеристики:

```

<!ATTLIST элемент
  атрибут тип характеристика
  атрибут тип характеристика
  ... >

```

Значения характеристики атрибутов приведены в таблице ниже:

Характеристика	Значение
#REQUIRED	Атрибут должен присутствовать в каждом экземпляре элемента в документе, является обязательным.
#IMPLIED	Атрибут может присутствовать, но необязателен.
#FIXED + значение по умолчанию	Атрибут всегда должен иметь только значение, предлагаемое по умолчанию; если атрибут не указан, значение по умолчанию все равно предполагается анализатором.
значение по умолчанию	Если атрибут не приведен, значение по умолчанию предполагается анализатором. Если атрибут имеется, у него может быть другое значение.

В XML определено несколько типов для атрибутов. По существу, атрибуты всегда имеют строковое значение, однако различают 3 категории типов атрибутов XML:

- строковые атрибуты (string attributes) - произвольная строка
- маркированные атрибуты (tokenized attributes) - строка определенного вида
- перечислимые атрибуты (enumerated attributes) - список значений

Тип	Категория	Значение
CDATA	строковый	Символьные данные (строка). Атрибуты данного типа могут принимать любое значение.
ID	маркированный	Имя, уникальное для данного документа. Значение атрибута данного типа всегда должно быть корректным именем: начинаться с буквы или символа подчеркивания, за которым следует последовательность 0 или более букв, цифр или знаков подчеркивания. Если 2 или более элементов имеют одинаковый ID, документ считается недействительным (invalid).
IDREF	маркированный	Ссылка на некоторый элемент, с атрибутом ID, который имеет то же значение, что и атрибут IDREF. Если элемент с указанным ID в документе отсутствует, документ считается недействительным.
IDREFS	маркированный	Несколько атрибутов IDREF, разделенных пустыми пространствами.
ENTITY	маркированный	Имя заранее определенной сущности. Если в DTD не определена указанная сущность, документ считается недействительным
ENTITIES	маркированный	Несколько имен сущностей (ENTITY), разделенных пустыми пространствами.
NMTOKEN	маркированный	Имя. Данный атрибут, как и ID, должен быть корректным именем, но его уникальность не требуется.

NMTOKENS	маркированный	Некоторые имена NMTOKEN, разделенных пустыми пространствами.
NOTATION	маркированный	Принимает одно из набора имен, указывающих на типы нотаций в определении DTD (см. ENTITY).
[список значений]	перечислимый	Принимает одно из серии значений, явным образом определенных пользователем.

Давайте на примере проектирования DTD книжного каталога рассмотрим описание атрибутов в DTD. Пусть основными элементами книжного каталога являются **book** и **author**. Именно на их примере мы и рассмотрим описание атрибутов.

CDATA

Примером строкового атрибута может быть, например, название книги:

Описание в DTD:

```
<!DOCTYPE books [
  <!ELEMENT book EMPTY>
  <!ATTLIST book
    title CDATA #REQUIRED>
]>
```

Применение в XML:

```
<book title="XML для начинающих" />
```

ID

Разовьем далее описание атрибутов элемента **book**. Каждая книга в каталоге должна иметь уникальный код. Это можно задать в DTD следующим образом:

Описание в DTD:

```
<!DOCTYPE books [
  <!ELEMENT book EMPTY>
  <!ATTLIST book
    code ID #REQUIRED
    title CDATA #REQUIRED>
]>
```

Применение в XML:

```
<book title="XML для начинающих" code="BK_5676" />
```

Теперь, если найдется две книги в каталоге, имеющие одинаковое значение атрибута **code**, документ будет недействительным.

IDREF

Теперь представим следующую ситуацию: авторы книг описываются отдельными элементами, каждый автор имеет уникальный идентификатор. В этом случае для связи книг с их авторами можно использовать атрибут типа **IDREF**:

Описание в DTD:

```

<!DOCTYPE books [
  <!ELEMENT author EMPTY>
  <!ELEMENT book EMPTY>
  <!ATTLIST author
    id ID #REQUIRED
    name CDATA #REQUIRED
    email CDATA #IMPLIED>
  <!ATTLIST book
    code ID #REQUIRED
    title CDATA #REQUIRED
    author IDREF #REQUIRED>
]>
Применение в XML:
<author id="AV_5698" name="Иванов И.И." />
<author id="AV_7655" name="Петров П.П." />
<book code="BK_5676" title="XML для начинающих" author="AV_7655" />

```

В данном примере у элемента **book** есть атрибут **author_id** типа **IDREF**, который может принимать значения, соответствующие существующему уникальному идентификатору автора. Обратите внимание: практически все атрибуты в примере объявлены с директивой **#REQUIRED** - обязательный, кроме атрибута **email** элемента **author** - он является необязательным - **#IMPLIED** (эта информация не критична, у автора может и не быть адреса электронной почты, или он по каким либо причинам отказался его предоставить).

IDREFS

В рассмотренном выше примере каждая книга может иметь только одного автора. Однако жизненный опыт показывает, что существует множество книг, написанных в соавторстве (2, 3 и более авторов). Как адаптировать наше DTD к такому случаю? Здесь целесообразно применить атрибут типа **IDREFS**, дающий возможность указать несколько ID, разделенных пустыми пространствами:

```

Описание в DTD:
<!DOCTYPE books [
  <!ELEMENT author EMPTY>
  <!ELEMENT book EMPTY>
  <!ATTLIST author
    id ID #REQUIRED
    name CDATA #REQUIRED
    email CDATA #IMPLIED>
  <!ATTLIST book
    code ID #REQUIRED
    title CDATA #REQUIRED
    authors IDREFS #REQUIRED>
]>
Применение в XML:
<author id="AV_5698" name="Иванов И.И." />
<author id="AV_9812" name="Сидоров В.П." />
<author id="AV_7655" name="Петров П.П." />
<book code="BK_5676" title="XML для начинающих" authors="AV_7655
AV_5698" />

```

```
<book code="BK_5677" title="XML для профессионалов" authors="AV_9898 AV_9812" />
```

В данном примере ссылка на несуществующий ID автора AV_9898, делающая документ в целом недействительным.

NMTOKEN

Атрибут данного типа должен быть корректным именем. По синтаксису он единичен ID, но не обязан быть уникальным в документе, или IDREF, но не должен ссылаться на действительный ID. В примере DTD книжного каталога это может быть название категории, к которой принадлежит книга (например: **markup** - разметка, **programming** - программирование и т.д.):

Описание в DTD:

```
<!DOCTYPE books [  
  <!ELEMENT author EMPTY>  
  <!ELEMENT book EMPTY>  
  <!ATTLIST author  
    id ID #REQUIRED  
    name CDATA #REQUIRED  
    email CDATA #IMPLIED>  
  <!ATTLIST book  
    code ID #REQUIRED  
    title CDATA #REQUIRED  
    authors IDREFS #REQUIRED>  
    category NMTOKEN #REQUIRED>  

```

Применение в XML:

```
<author id="AV_5698" name="Иванов И.И." />  
<author id="AV_9812" name="Сидоров В.П." />  
<author id="AV_7655" name="Петров П.П." />  
<author id="AV_6789" name="Горбачев М.С." />  
<book code="BK_7644" title="Язык Разметки Гипертекста" category="markup"  
authors="AV_7655 AV_5698" />  
<book code="BK_3245" title="C++ для профессионалов" category="programming"  
authors="AV_9812" />  
<book code="BK_3245" title="Серверные сценарии - ASP" category="server side  
scripting" authors="AV_6789" />
```

Ошибка, сделанная в примере: "server side scripting" не является корректным **именем** с точки зрения анализатора XML.

Список значений

Атрибуты данного типа принимают одно из указанного списка значений. Переделаем наш пример так, чтобы категория, к которой принадлежит книга, была перечислимой:

Описание в DTD:

```
<!DOCTYPE books [  
  <!ELEMENT author EMPTY>  
  <!ELEMENT book EMPTY>  
  <!ATTLIST author  
    id ID #REQUIRED
```

```

    name CDATA #REQUIRED
    email CDATA #IMPLIED>
<!ATTLIST book
    code ID #REQUIRED
    title CDATA #REQUIRED
    authors IDREFS #REQUIRED>
    category ("common"|"markup"|"programming"|"scripting") "common">
]>
Применение в XML:
<author id="AV_5698" name="Иванов И.И." />
<author id="AV_9812" name="Сидоров В.П." />
<author id="AV_5566" name="Орлов П.П." />
<author id="AV_6789" name="Горбачев М.С." />
<book code="BK_7644" title="Язык Разметки Гипертекста" category="markup"
authors="AV_7655 AV_5698" />
<book code="BK_3245" title="C++ для профессионалов" category="programming"
authors="AV_9812" />
<book code="BK_6677" title="Бизнес и компьютер" authors="AV_5566" />

```

В данном примере атрибут **category** принимает значения из заданного списка. Кроме того, значением по умолчанию для данного атрибута является **"common"** (выделен в коде зеленым фоном). Таким образом, если атрибут **category** не указан, значение по умолчанию будет подставлено анализатором. Если значение атрибута **category** не входит в указанный список, документ будет считаться недействительным.

Параметрические сущности (продолжение)

Параметрические сущности используются для сокращения и упрощения записи часто встречающихся в DTD фрагментов. Представим себе следующую ситуацию: в нашем каталоге могут содержаться как книги, так и периодические издания (журналы, газеты). И у одних и у вторых есть такие атрибуты, как код, название, год издания, типография и т.п. в таком случае DTD примет следующий вид (DTD приводится в сокращенном виде):

```

<!DOCTYPE catalog [
...
<!ELEMENT book EMPTY>
<!ELEMENT magazin EMPTY>
<!ATTLIST book
    code ID #REQUIRED
    title CDATA #REQUIRED
    year CDATA #REQUIRED
    publisher CDATA #REQUIRED
    category ("common"|"markup"|"programming"|"scripting") "common"
    authors IDREFS #REQUIRED>
<!ATTLIST magazin
    code ID #REQUIRED
    title CDATA #REQUIRED
    year CDATA #REQUIRED
    publisher CDATA #REQUIRED
    number CDATA #REQUIRED>
]>

```

Ну, например, так. Как видно из примера в списках атрибутов для книги и журнала присутствуют повторяющиеся элементы (**code**, **title**, ...). Оптимизировать данное объявление можно путем создания и использования такой параметрической сущности:

```
<!ENTITY % commonParams
"code ID #REQUIRED
title CDATA #REQUIRED
year CDATA #REQUIRED
publisher CDATA #REQUIRED">
<!ELEMENT book EMPTY>
<!ELEMENT magazin EMPTY>
<!ATTLIST book
%commonParams;
category ("common"|"markup"|"programming"|"scripting") "common"
authors IDREFS #REQUIRED>
<!ATTLIST magazin
%commonParams;
number CDATA #REQUIRED>
```

Необходимо отметить, что параметрические сущности могут использоваться только во внешних DTD.

W3C парсеры для XML.

Проблемой при валидации (проверка правильности XML документа согласно схеме) является тот факт, что соответствие документов их схемам некоторые браузеры не проверяют. В связи с этим возникает необходимость использовать возможности DOM (Document Object Model) для проверки правильности. Функция валидации так или иначе присутствует в любом парсере. Ее необходимо правильно вызвать.

Проверить соответствие XDR схеме в ОС Windows несколько легче, поскольку XML парсер от Microsoft является частью операционной системы. Достаточно воспользоваться Java скриптом для проверки (как в прошлой лабораторной работе).

XSD является стандартом, поддерживаемым и развиваемым консорциумом W3C. В рамках этой поддержки The Apache Software Foundation создала набор ПО, представляющего собой парсеры и другое обеспечение для работы с XML. Одним из таких известных парсеров является Xerces. Он существует в виде отдельного ПО, реализованного на C++ или Java. Чтобы не ограничивать Вас в выборе инструментальной среды и ОС, будем использовать Java реализацию ввиду ее кроссплатформенности и простоты использования.

Замечание. Для запуска Java приложения необходимо, чтобы на компьютере была установлена Java машина от Sun. Желательно с Java SDK.

Xerces — семейство программных пакетов для синтаксического анализа и работы с XML можно скачать с официального сайта <http://xerces.apache.org/>.

Задание на лабораторную работу

Необходимо для XML документа определить его структуру с помощью DTD (внешней). Осуществить проверку соответствия документа описанию его структуры.

Проверку можно осуществить с помощью специального валидатора.

Для демонстрации правильности DTD необходимо проверить не только прохождение валидации, но и ее непрохождение (т.е. смоделировать несолько ошибок, которые явно указывают на то или иное описание в DTD).

Требование к отчету:

- титульный лист, номер и название лабораторной работы;
- цель работы, задание на работу;
- листинг документа XML (файла);
- скриншот результатов работы программы валидатора;
- результат проверки XML документа валидатором;
- выводы по работе;
- ответы на контрольные вопросы (при наличии).